FUNCTIONS & LAMBDA

Programming for Data Science

Functions

- In Python a function is defined using the *def* keyword

```
def my_function():
  print("Hello from a function")

my_function()
```

Arguments

Arguments are often shortened to args in Python documentation.

```
def my_function(fname):
  print(fname + " Refsnes")

my_function("Emil")
my_function("Tobias")
my_function("Linus")
```

Output

```
Emil Refsnes
Tobias Refsnes
Linus Refsnes
```

Arguments

```
def my_function(fname):
  print(fname + " Refsnes")

my_function("Emil")
my_function("Tobias")
my_function("Linus")
```

```
def my_function(fname, lname):
  print(fname + " " + lname)

my_function("Emil", "Refsnes")
```

```
def my_function(fname, lname):
  print(fname + " " + lname)

my_function("Emil")
```

Arbitrary Arguments, *args

```
def my_function(*kids):
  print("The youngest child is " + kids[2])

my_function("Emil", "Tobias", "Linus")
```

Arbitrary Arguments, *args

This way the function will receive a tuple of arguments, and can access the items accordingly

```
def my_function(*kids):
  print("The youngest child is " + kids[2])

my_function("Emil", "Tobias", "Linus")
```

Output

```
The youngest child is Linus
```

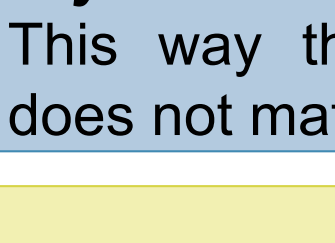
Keyword Arguments

```
def my_function(child3, child2, child1):
  print("The youngest child is " + child3)

my_function(child1 = "Emil", child2 = "Tobias", child3 = "Linus")
```

Keyword Arguments

We can send arguments with the ***key = value*** syntax.
This way the order of the arguments does not matter.



```
def my_function(child3, child2, child1):
  print("The youngest child is " + child3)

my_function(child1 = "Emil", child2 = "Tobias", child3 = "Linus")
```

Output

```
The youngest child is Linus
```

Arbitrary Keyword Arguments, **kwargs

```
def my_function(**kid):
  print("His last name is " + kid["lname"])

my_function(fname = "Tobias", lname = "Refsnes")
```

Arbitrary Keyword Arguments, **kwargs

This way the function will receive a dictionary of arguments, and can access the items accordingly.

```
def my_function(**kid):
  print("His last name is " + kid["lname"])

my_function(fname = "Tobias", lname = "Refsnes")
```

Output

```
His last name is Refsnes
```

Default Parameter Value

```
def my_function(country = "Norway"):
  print("I am from " + country)

my_function("Sweden")
my_function("India")
my_function()
my_function("Brazil")
```

Output

```
I am from Sweden
I am from India
I am from Norway
I am from Brazil
```

Passing a List as an Argument

```
def my_function(food):
  for x in food:
    print(x)

fruits = ["apple", "banana", "cherry"]

my_function(fruits)
```

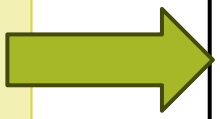
Output

```
apple
banana
cherry
```

Return & Pass

```
def my_function(x):
  return 5 * x

print(my_function(3))
print(my_function(5))
print(my_function(9))
```



```
Output
15
25
45
```

```
def myfunction():
  pass
```



function definitions cannot be empty, but if we for some reason have a function definition with no content, put in the ***pass*** statement to avoid getting an error.

Recursion

```
def tri_recursion(k):
  if(k > 0):
    result = k + tri_recursion(k - 1)
    print(result)
  else:
    result = 0
  return result

print("Recursion Example Results:")
tri_recursion(6)
```

Output

```
Recursion Example Results:
1
3
6
10
15
21
```

Lambda

- A lambda function is a small anonymous function.
- A lambda function can take any number of arguments, but can only have one expression.

```
lambda arguments : expression
```

Lambda

```
x = lambda a : a + 10
print(x(5))
```



Output
15

```
x = lambda a, b : a * b
print(x(5, 6))
```



Output
30

```
x = lambda a, b, c : a + b + c
print(x(5, 6, 2))
```



Output
13

Why Lambda?

```
def myfunc(n):
  return lambda a : a * n

mydoubler = myfunc(2)
mytripler = myfunc(3)

print(mydoubler(11))
print(mytripler(11))
```

Output

```
22
33
```

Thank you!